

PROPUESTA TÉCNICA DE DESARROLLO DE SOFTWARE

Antioquia Biodiversa

Versión: v2.0 — Propuesta técnica unificada

Gobernación de Antioquia — Secretaría de Ambiente

Junio 2026

1. Introducción a la Solución

Antioquia Biodiversa es una aplicación web mobile-first que centraliza la información sobre la biodiversidad del departamento de Antioquia, accesible desde cualquier dispositivo mediante código QR, sin necesidad de instalación. La solución incluye módulos de biodiversidad, recursos hídricos y programas comunitarios (Jóvenes pa' Lante, Guarda Cuencas, Especie del Mes), galerías fotográficas mensuales para JPL y Guarda Cuencas, un panel de administración web para curadores de contenido, y una API REST documentada. Está construida sobre un stack tecnológico moderno de código abierto, optimizado para bajo costo de operación, alta accesibilidad ciudadana y mantenibilidad a largo plazo por el equipo institucional de la Gobernación.

2. Detalles Técnicos

2.1. Metodología de Desarrollo

- Se utilizará una metodología ágil incremental con entregas quincenales sujetas a revisión y aprobación del líder funcional de la Gobernación.
- Fases del ciclo: Análisis y Diseño → Construcción (sprints) → Pruebas QA → Transición a producción → Capacitación
- Gestión de tareas: Azure DevOps institucional (repositorio Git + tablero de seguimiento)
- Entregables: quincenales con acta de aprobación por fase
- Control de versiones: GitFlow con ramas main (producción estable) y develop (integración continua)
- Convención de commits: Conventional Commits (feat:, fix:, docs:, refactor:) para mantener historial legible y auditable

2.2. Stack Tecnológico (v2.0)

La siguiente tabla refleja el stack tecnológico actualizado, incluyendo los componentes nuevos incorporados en la versión 2.0 de esta propuesta.

Capa	Tecnología	Versión	Estado
Frontend	HTML5 + CSS3 + JavaScript Vanilla	—	Sin cambios — máxima compatibilidad, sin frameworks, carga rápida en red móvil
Backend	Node.js + Express.js	22 LTS / 4.x	Sin cambios — open source, alto rendimiento, en catálogo oficial de la Gobernación
Base de Datos	MongoDB 7.x + Mongoose ODM	7.x	Sin cambios — modelo documental óptimo para catálogo de especies
Caché	Redis + ioredis	7.x / ^5	Nuevo (v2.0) — capa de caché en memoria para datos estáticos
Autenticación Admin	Microsoft Entra ID (OAuth 2.0 + OIDC)	MSAL Node v3	Nuevo (v2.0) — reemplaza express-session + bcrypt

Logs	Winston	latest	Nuevo (v2.0) — logs JSON estructurados con traceld
SAST — capa 1	ESLint-security (eslint-plugin-security)	plugin npm	Nuevo (v2.0) — filtro rápido de patrones peligrosos Node.js
SAST — capa 2	Semgrep Community (p/nodejs, p/owasp-top-ten)	CLI gratuito	Nuevo (v2.0) — análisis OWASP Top 10 en el pipeline CI
CI/CD	Azure DevOps (plantillas TI Gobernación)	—	Nuevo (v2.0) — pipeline institucional
Optimización Imágenes	sharp 0.34 (Node.js)	^0.34	Sin cambios — conversión automática JPG/PNG a WebP
Carga de Archivos	multer (Node.js)	^1.4	Sin cambios — uploads multipart/form-data
Servidor Web	Nginx + PM2	latest	Sin cambios — proxy inverso, SSL, reinicio automático
Control de Versiones	Git + Azure DevOps	—	Sin cambios — plataforma institucional
Infraestructura	Ubuntu Server 24.04 LTS	24.04	Sin cambios — soporte LTS hasta abril 2029

3. Arquitectura General

3.1. Diagrama de Arquitectura

- Estilo: Monolito Modular (recomendado en los lineamientos de la Gobernación para aplicaciones de dominio unificado)
- Patrón de diseño backend: Clean Architecture — separación entre Dominio (lógica de negocio), Aplicación (casos de uso) e Infraestructura (Express/MongoDB/Redis)
- Comunicación: API RESTful bajo protocolo HTTPS con contratos Swagger/OpenAPI definidos antes de la codificación (API First)
- Seguridad: TLS 1.2+ (Certbot/Let's Encrypt), validación de entradas en todos los endpoints, cumplimiento OWASP Top 10 verificado por pipeline SAST
- Autenticación: Microsoft Entra ID vía OAuth 2.0 + OIDC para el panel de administración
- Caché: Redis 7 en modo standalone entre Express y MongoDB para datos casi estáticos
- Monitoreo: Winston con logs JSON estructurados y endpoint /api/health para verificación de estado del servicio
- Entornos: Desarrollo (Dev) → QA/Staging (réplica de producción) → Producción (solo tras aprobación de QA)
- Accesibilidad: La interfaz pública cumple con los criterios WCAG 2.1 Nivel AA (contraste de colores, navegación por teclado, compatibilidad con lectores de pantalla). Validado con la herramienta WAVE antes de cada pase a producción.
- Privacidad (Ley 1581): La política de tratamiento de datos personales es visible en la aplicación y cuenta con mecanismo de aceptación explícito por parte del usuario.

Flujo de petición del ciudadano:

Ciudadano (móvil / escritorio) → [HTTPS] → Nginx (proxy + SSL) → Redis (caché)
→ Node.js / Express + PM2 → MongoDB Atlas

3.2. Requisitos de Despliegue

Componente	Especificación
Servidor de Aplicaciones	Ubuntu Server 24.04 LTS — 4 vCPU, 16 GB RAM
Almacenamiento	2 TB (aplicación + fotos de especies + base de datos)
Base de datos	MongoDB Atlas (tier a definir por TI Gobernación)
Dominio y SSL	Dominio institucional .gov.co con certificado SSL (Certbot/Let's Encrypt)
Entornos	Al menos tres ambientes lógicos independientes: Desarrollo, QA/Staging y Producción
Redis	256 MB RAM dedicados en el mismo servidor (modo standalone)

4. Ajustes Técnicos Propuestos (v2.0)

Esta sección recoge los ajustes técnicos incorporados en la versión 2.0 de la propuesta. El stack base (Node.js 22 LTS, MongoDB Atlas, arquitectura modular con Express) se mantiene sin cambios. Los cuatro ajustes corresponden a mejoras en seguridad, rendimiento y observabilidad, incorporando las respuestas recibidas del equipo de TI de la Gobernación.

Esta versión incorpora dos precisiones frente a la propuesta original:

- **CI/CD on-premises:** El pipeline CI se configurará usando las plantillas ya generadas por el equipo de TI. El CD depende del esquema de gestión de servidores on-premises de la Gobernación.
- **SAST en dos capas:** El análisis estático de seguridad (SAST) se implementa en dos capas: ESLint-security (filtro rápido) y Semgrep Community con reglas OWASP Top 10 (análisis profundo).

Resumen ejecutivo de ajustes

Ajuste	RNF impactado	Mejora concreta
1. Microsoft Entra ID	RNF05, RNF08	Elimina contraseñas en la BD. MFA automático. Revocación inmediata al salir de la institución.
2. Caché con Redis 7	RNF02, RNF06	Catálogo en <5 ms. MongoDB libre para escrituras bajo carga alta (eventos QR masivos).
3. Monitoreo (Winston + /api/health)	RNF06, RNF07	Logs estructurados con trazabilidad por petición. Estado del servicio verificable en cualquier momento.
4. SAST (ESLint-security + Semgrep)	RNF05	OWASP Top 10 verificado automáticamente en cada push. No depende de revisión manual.

4.1. Autenticación con Microsoft Entra ID

Situación actual

El panel de administración utilizaba express-session con bcrypt. Las contraseñas de los curadores se almacenaban en MongoDB Atlas, lo que exponía credenciales ante un eventual compromiso de la base de datos y dejaba la gestión del ciclo de vida de usuarios bajo responsabilidad del contratista durante toda la garantía.

Propuesta

Integrar el panel de administración con Microsoft Entra ID (Azure Active Directory institucional). Los curadores inician sesión con su cuenta corporativa @antioquia.gov.co. La aplicación nunca almacena ni verifica contraseñas: solo valida el token que emite Microsoft.

Justificación

- **Infraestructura existente:** La Gobernación ya tiene Entra ID con todos sus funcionarios registrados. No se monta un sistema de identidad paralelo.
- **MFA sin desarrollo extra:** El MFA vía Microsoft Authenticator, ya activo para los funcionarios, se hereda sin costo adicional de desarrollo.
- **Revocación instantánea:** Cuando un curador sale de la institución, TI desactiva su cuenta en Entra ID y pierde el acceso al panel de forma inmediata y automática.
- **Superficie de ataque reducida:** La aplicación nunca toca una contraseña. Si la BD es comprometida, no hay credenciales de acceso al panel que extraer.

Especificación técnica

Componente	Detalle
Protocolo	OAuth 2.0 + OpenID Connect (OIDC)
Flujo de autorización	Authorization Code Flow con PKCE
Librería backend (Node.js)	@azure/msal-node + passport-azure-ad
Librería frontend (panel admin)	@azure/msal-browser
Sesión	JWT emitido por Entra ID — sin contraseñas en la aplicación
MFA	Heredado del tenant institucional (Microsoft Authenticator)
Roles de aplicación	Curador.Biodiversidad · Curador.GuardaCuentas · Admin.Contenido
Definición de roles	App Registration en el tenant de la Gobernación

Prerequisitos — acción requerida por la Gobernación:

- Client ID de la aplicación registrada en el tenant de Entra ID.
- Tenant ID del directorio institucional.
- Configuración de los redirect_uri para QA y Producción.
- Definición de los roles en el App Registration: Curador.Biodiversidad, Curador.GuardaCuentas, Admin.Contenido.

4.2. Caché con Redis 7

Situación actual

Cada solicitud al catálogo de especies generaba una consulta directa a MongoDB Atlas (60–120 ms). En eventos de alto tráfico como el escaneo masivo de QR, las peticiones simultáneas saturaban la base de datos innecesariamente, dado que el catálogo cambia únicamente cuando el administrador realiza una importación masiva.

Propuesta

Implementar Redis 7 como capa de caché entre Express y MongoDB usando ioredis. La primera consulta llena el caché; las subsiguientes responden desde memoria RAM en menos de 5 ms. Redis corre en el mismo servidor Ubuntu en modo standalone, sin costo de infraestructura adicional.

Estrategia de TTL e invalidación

Recurso / Endpoint	TTL	Evento de invalidación
/api/especies (catálogo completo)	30 minutos	El admin ejecuta importación masiva
/api/especies/:id (ficha individual)	60 minutos	Se actualiza esa especie en la BD
/api/subregiones y /api/grupos	24 horas	Dato estático — invalidación manual
Galería JPL / Guarda Cuencas	10 minutos	Se publica el JSON mensual desde el panel
Especie del Mes	1 hora	Se actualiza desde el panel de administración

Especificación técnica

Componente	Detalle
Software	Redis 7.x (paquete oficial de Ubuntu)
Librería Node.js	iredis ^5
Modo de despliegue	Standalone en el servidor de aplicación (sin cluster)
Memoria máxima	256 MB con política de evicción allkeys-lru
Persistencia	RDB snapshot cada 15 min (tolerancia a reinicios)

4.3. Monitoreo y Observabilidad

Situación actual

Los únicos registros disponibles eran los logs de PM2 en archivos de texto plano. Sin estructura ni trazabilidad, el diagnóstico de errores era lento y dependía de la memoria del desarrollador.

Propuesta

Componente	Función
Winston	Logs JSON estructurados con nivel, módulo, timestamp y traceId único por petición. Facilita el diagnóstico rápido de errores en producción.
GET /api/health	Endpoint que verifica el estado de MongoDB y Redis. Usado por Nginx como upstream health check para detectar caídas automáticamente.

4.4. Análisis Estático de Seguridad (SAST) en Azure DevOps

Situación actual

El RNF05 establecía cumplimiento con el OWASP Top 10, pero no existía ningún mecanismo automatizado que lo verificara. La detección de vulnerabilidades dependía exclusivamente de la revisión manual durante el pull request, que no es sistemática ni reproducible.

Propuesta — dos herramientas complementarias

Se integran ESLint-security y Semgrep Community en el pipeline de Azure DevOps. Las dos herramientas se ejecutan automáticamente en cada push y el pipeline falla si se detecta una vulnerabilidad de severidad Alta o Crítica.

Herramienta	Rol en el pipeline	Alcance	Costo
ESLint-security	Filtro rápido — corre en segundos, falla el build de inmediato.	Patrones peligrosos específicos de Node.js/Express: eval() con input de usuario, path traversal, prototype pollution, ReDoS.	Gratuito. Plugin npm.
Semgrep Community	Análisis OWASP profundo — corre después de ESLint.	Reglas p/nodejs y p/owasp-top-ten. Cubre las 10 categorías OWASP para Node.js/JavaScript.	Gratuito. Binario CLI.

Flujo en el pipeline de CI

- **Paso 1:** Checkout del código fuente desde Azure Repos.
- **Paso 2:** npm install — instalación de dependencias.
- **Paso 3:** npm audit — análisis de dependencias de terceros. Falla el pipeline si detecta vulnerabilidades conocidas de severidad Alta o Crítica (CVEs).
- **Paso 4:** npm run lint — ESLint con eslint-plugin-security. Falla el pipeline si detecta una regla de severidad error.
- **Paso 5:** npm test — ejecución de pruebas unitarias (Jest).
- **Paso 6:** Semgrep scan — análisis con reglas p/nodejs y p/owasp-top-ten. Falla el pipeline si detecta vulnerabilidad de severidad Alta o Crítica.
- **Paso 7:** (Solo en rama develop/main) Build y empaquetado para despliegue.

Prerrequisitos y plan de implementación

Prerrequisitos por parte de la Gobernación

Prerrequisito	Para qué ajuste	Responsable	Estado
Crear proyecto en Azure DevOps y proveer acceso al repositorio	Todos	TI Gobernación	Pendiente
Proveer Client ID, Tenant ID y configurar redirect_uri en Entra ID	Ajuste 1 — Entra ID	TI Gobernación	Pendiente
Definir roles en App Registration	Ajuste 1 — Entra ID	TI Gobernación	Pendiente

(Curador.Biodiversidad, etc.)			
Confirmar VM/servidor on-premises y credenciales de despliegue	Ajustes 2, 3 y CD	TI Gobernación	Pendiente

Secuencia de implementación sugerida

Fase	Ajuste	Duración estimada	Dependencia
Fase 0 — Infraestructura	Azure DevOps: repositorio, tableros, plantillas CI	1 semana	Acceso a Azure DevOps
Fase 1 — Calidad de código	Ajuste 4: ESLint-security + Semgrep en pipeline CI	2–3 días	Acceso a Azure DevOps
Fase 2 — Rendimiento	Ajuste 2: Redis 7 + ioredis en el backend	3–4 días	VM/servidor definido
Fase 2 — Observabilidad	Ajuste 3: Winston + endpoint /api/health	1 día	VM/servidor definido
Fase 3 — Autenticación	Ajuste 1: Microsoft Entra ID	1–2 semanas	Client ID + Tenant ID de Gobernación

Nota: Los ajustes 2 (Redis) y 3 (Monitoreo) pueden implementarse en paralelo una vez que el servidor on-premises esté disponible. El ajuste 3 requiere solo 1 día al haberse simplificado a Winston + /api/health.

5. Soporte y Mantenimiento

5.1. Período de Garantía

Se ofrece un período de garantía de 6 meses contados a partir de la firma del acta de entrega final.

- Cobertura: Corrección de errores (bugs) imputables al código desarrollado.
- Exclusiones: Nuevas funcionalidades, cambios en reglas de negocio posteriores a la aprobación, fallos ocasionados por infraestructura de terceros o mal uso del sistema.

5.2. Mantenimiento Evolutivo (Opcional)

Finalizado el período de garantía, cualquier soporte o nueva funcionalidad se cotizará por hora de desarrollo o mediante contrato de mantenimiento mensual.

6. Cronograma de Ejecución

Las fechas de inicio y fin se definirán una vez se formalice el contrato y TI Gobernación confirme el acceso a la infraestructura institucional (servidor on-premises, Azure DevOps, MongoDB Atlas). El cronograma detallado será acordado en la reunión de arranque (kick-off).

Fase / Módulo	Actividad Principal	Fecha Inicio	Fecha Fin	Entregable
Fase 0	Configuración	[PENDIENTE]	[PENDIENTE]	Ambientes

	Azure DevOps, servidor, BD y entornos (Dev / QA / Prod)			operativos
Fase 1	Pipeline CI: ESLint-security + Semgrep.	[PENDIENTE]	[PENDIENTE]	Pipeline SAST activo.
Módulo Biodiversidad	Despliegue y validación del módulo de especies en producción	[PENDIENTE]	[PENDIENTE]	Módulo en producción
Módulos Comunidad	Despliegue de Agua, JPL, Guarda Cuencas, Especie del Mes y panel admin	[PENDIENTE]	[PENDIENTE]	Módulos en producción
Fase 2	Redis + Winston + endpoint /api/health	[PENDIENTE]	[PENDIENTE]	Caché activa. Logs estructurados.
QA	Pruebas integrales (Jest, Postman/Newman, WAVE WCAG, SAST), correcciones y validación de seguridad	[PENDIENTE]	[PENDIENTE]	Informe de QA
Fase 3	Integración Microsoft Entra ID para autenticación del panel admin	[PENDIENTE]	[PENDIENTE]	Panel admin con SSO institucional
Despliegue	Puesta en producción final y capacitación al equipo de curadores	[PENDIENTE]	[PENDIENTE]	Sistema en vivo + acta de entrega

7. Diccionario de Definiciones y Acrónimos

Término	Definición
API	Interfaz de Programación de Aplicaciones.
Backend	Parte lógica del sistema que procesa los datos, no visible directamente por el usuario.
Clean Architecture	Patrón de diseño que separa el dominio de la infraestructura para mayor mantenibilidad (recomendado por la Gobernación).
Entra ID	Servicio de identidad de Microsoft (anteriormente Azure Active Directory). Gestiona autenticación y autorización con soporte nativo de MFA.
Frontend	Capa de presentación visible al usuario en el navegador.
GitFlow	Modelo de ramificación Git con ramas main

	(producción) y develop (integración continua).
JWT	JSON Web Token — formato estándar para transmitir información de autenticación entre partes de forma segura.
MFA	Multi-Factor Authentication — verificación de identidad mediante dos o más factores.
MongoDB	Base de datos NoSQL orientada a documentos.
Nginx	Servidor web y proxy inverso de código abierto.
OIDC	OpenID Connect — protocolo de autenticación sobre OAuth 2.0.
OWASP Top 10	Lista de las 10 vulnerabilidades de seguridad web más críticas, publicada por el Open Web Application Security Project.
PM2	Gestor de procesos para aplicaciones Node.js en producción.
QA	Quality Assurance — aseguramiento de calidad.
Redis	Base de datos en memoria usada como caché para reducir tiempos de respuesta.
SAST	Static Application Security Testing — análisis estático de seguridad del código fuente.
Semgrep	Herramienta de análisis estático de código que detecta vulnerabilidades de seguridad mediante reglas OWASP.
TLS	Transport Layer Security — protocolo de cifrado para comunicaciones web (requiere versión 1.2 o superior).
TTL	Time To Live — tiempo de vida de un dato en caché antes de ser invalidado.
WebP	Formato de imagen moderno con compresión superior a JPG/PNG.
Winston	Librería de logging para Node.js que genera logs estructurados en formato JSON.

8. Compromisos del Contratista

- Entrega del código fuente completo sin ofuscación en repositorio Azure DevOps institucional.
- Documentación técnica: README.md, Swagger/OpenAPI, manual de despliegue e instalación.
- Cumplimiento de la Guía de Arquitectura y Buenas Prácticas de la Gobernación de Antioquia.
- Accesibilidad WCAG 2.1 Nivel AA en la interfaz pública: contraste, navegación por teclado, compatibilidad con lectores de pantalla. Validación con WAVE antes de cada pase a producción.
- Uso de GitFlow (ramas main/develop) y Conventional Commits para control de versiones.
- Código y comentarios en español (neutro) conforme a los lineamientos institucionales.
- Uso exclusivo de librerías con licencias compatibles con uso institucional (MIT, Apache 2.0).
- Cumplimiento de la Ley 1581 (Habeas Data): política de tratamiento de datos personales visible en la aplicación con mecanismo de aceptación explícito (checkbox no pre-marcado).
- Cifrado en tránsito (TLS 1.2+) en todas las comunicaciones.
- Autenticación del panel admin mediante Microsoft Entra ID (OAuth 2.0 + OIDC); prohibido almacenar contraseñas en la BD.

- Estrategia de tres entornos: Desarrollo, QA/Staging y Producción; datos de prueba anonimizados en entornos no productivos.
- Pipeline SAST activo en Azure DevOps: ESLint-security + Semgrep (p/owasp-top-ten) en cada push.
- Análisis de dependencias de terceros (npm audit) en el pipeline CI: falla si detecta vulnerabilidades de severidad Alta o Crítica (CVEs).
- Monitoreo operativo activo: logs estructurados en formato JSON con Winston y endpoint /api/health para verificación continua del estado del servicio.
- Capacitación básica al equipo de curadores de la Secretaría de Ambiente para el uso del panel de administración.

9. Control de Ajustes

Versión	Fecha	Descripción del Cambio	Responsable
1.0	Junio 2026	Versión inicial.	Sebastián Guzmán Díaz
1.1	Junio 2026	Se agregan galerías JPL y Guarda Cuencas, panel de administración de curadores y stack sharp/multer. Alineación con lineamientos de la Gobernación: Clean Architecture, GitFlow, Conventional Commits, tres entornos y privacidad Ley 1581.	Sebastián Guzmán Díaz
2.0	Junio 2026	Incorporación de cuatro ajustes técnicos: (1) Microsoft Entra ID para autenticación del panel admin, (2) Redis 7 como capa de caché, (3) monitoreo con Winston y endpoint /api/health, (4) pipeline SAST con ESLint-security + Semgrep. Actualización del stack tecnológico, arquitectura y compromisos del contratista.	Sebastián Guzmán Díaz

10. Aprobación y Firmas

Rol	Nombre	Firma	Fecha
Contratista	Sebastián Guzmán Díaz		
Contratista	Alejandro López		
Líder Funcional Gobernación	Tatiana Rendón Arroyave		

